

Part-II Recommender Systems

Theory and Implementation

Dr. Mustansar Ali Ghazanfar

mghazanfar@uel.ac.uk

Learning Outcomes

By the end of this lecture on Recommender Systems, you will be able to:

- Define Matrix Factorisation
- Gradient Descent for RS
- Scalable approaches for RS
- Implement in pyspark (lab)



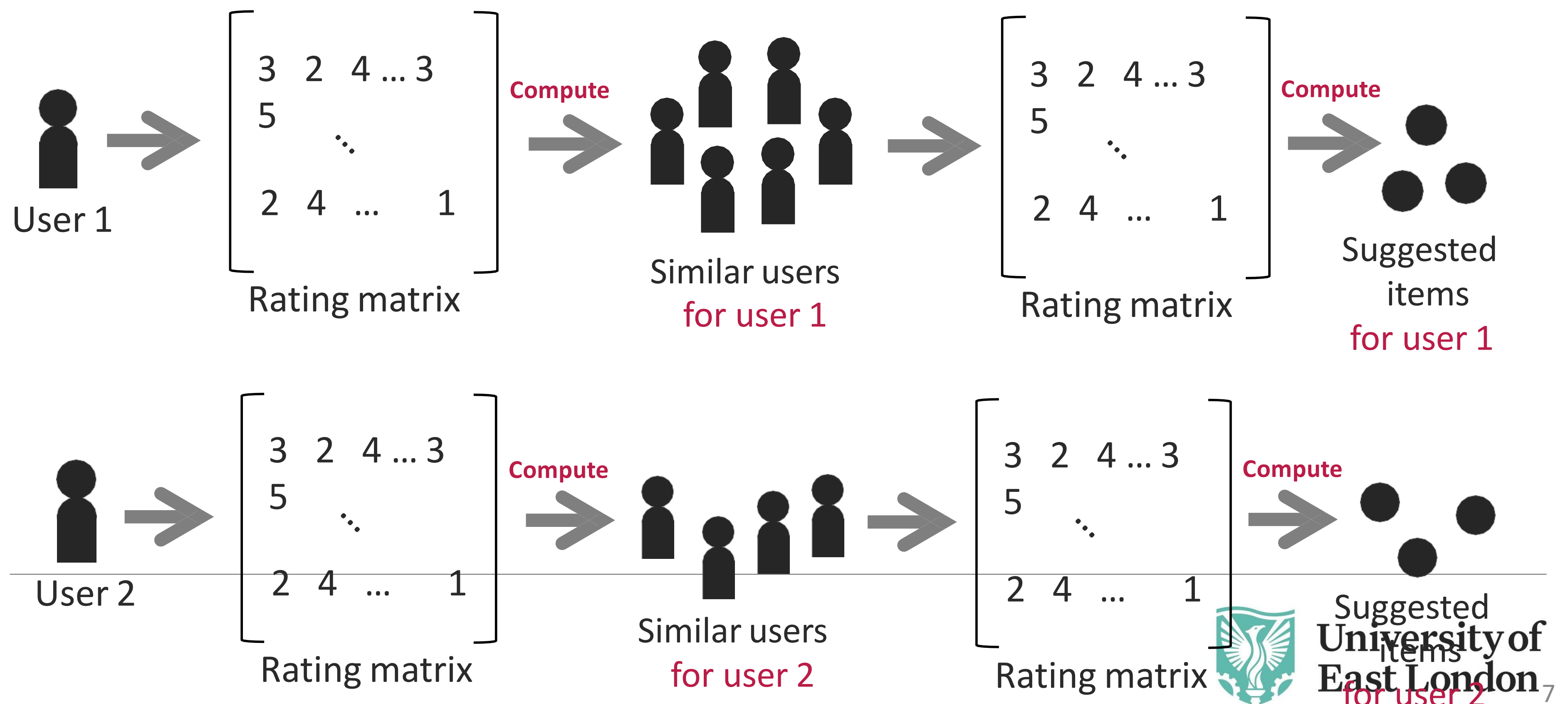
Matrix Factorization: Scalable Collaborative Filtering



1 Problems on Simple Collaborative Filtering

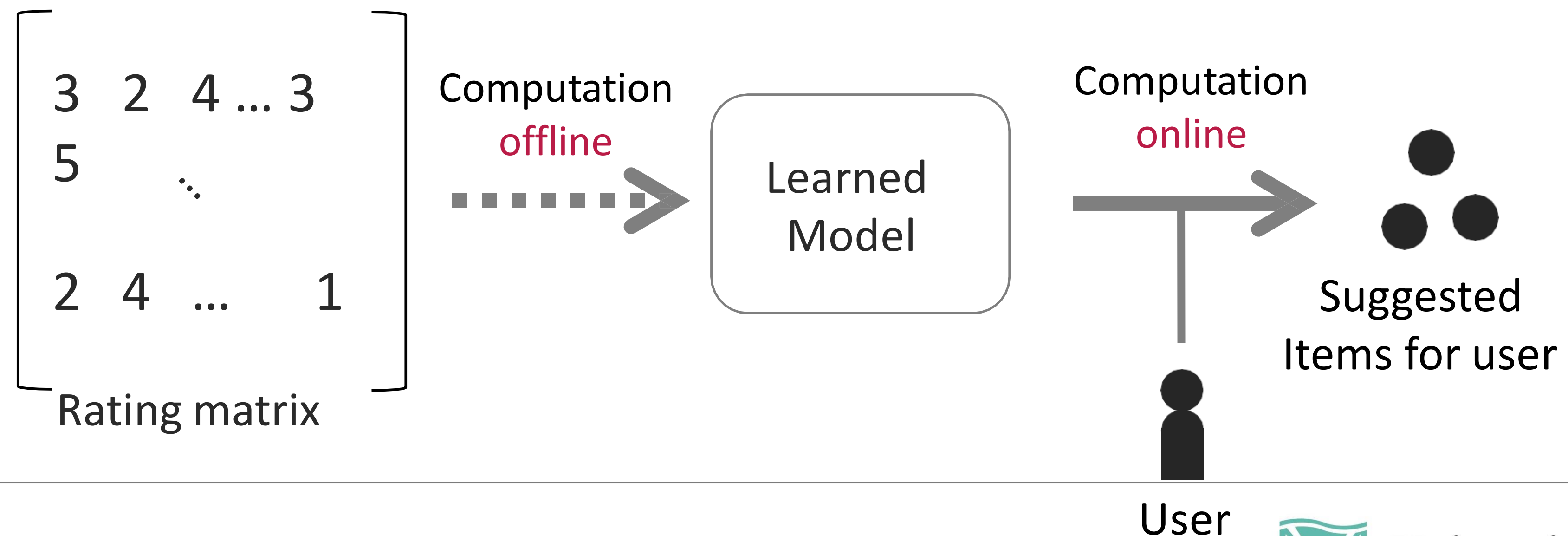
High computational cost

- The rating matrix is directly used every time systems find similar user/items and make predictions
- CF approaches do not scale for most real world scenarios



Model-based approach

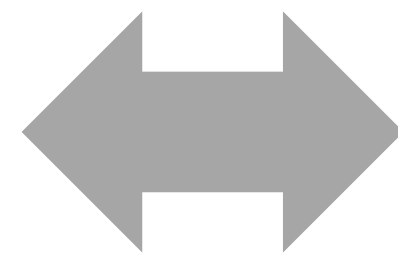
- Based on offline pre-processing
- At run-time, only the learned model is used for rating prediction
- Learned models can be updated



Memory-based approach vs. model-based approach

Memory-based approach

- User-based CF
- Item-based CF



Model-based approach

- Matrix factorization
- Alternative Least Square
- Singular Value Decomposition

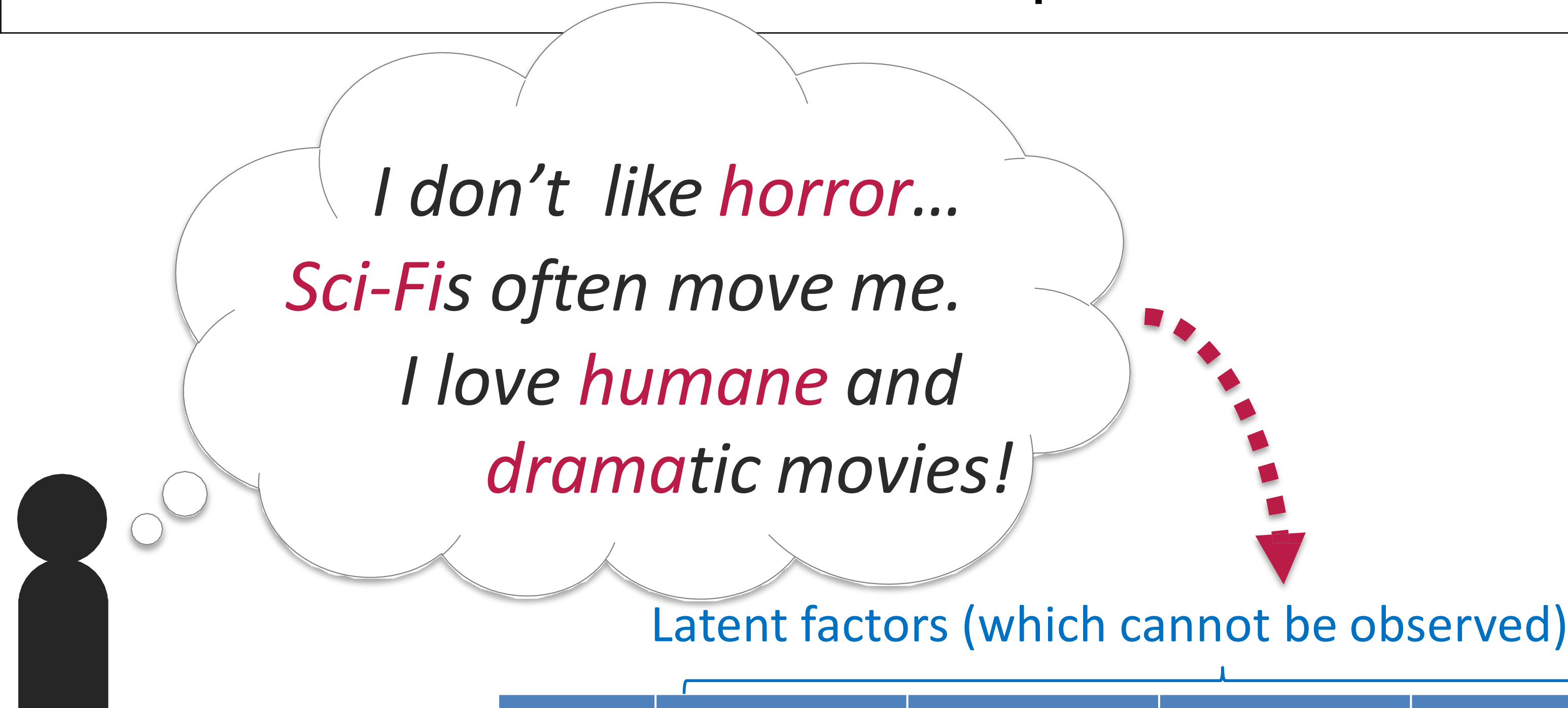


2 Matrix Factorization

Basic idea 1 (1/2)

Assumes that **latent factors** exist in users/items

User	God father	Terminator	Money game	Titanic	Back to the future	...	X-men
Alice	5	1	4	4	3	...	2

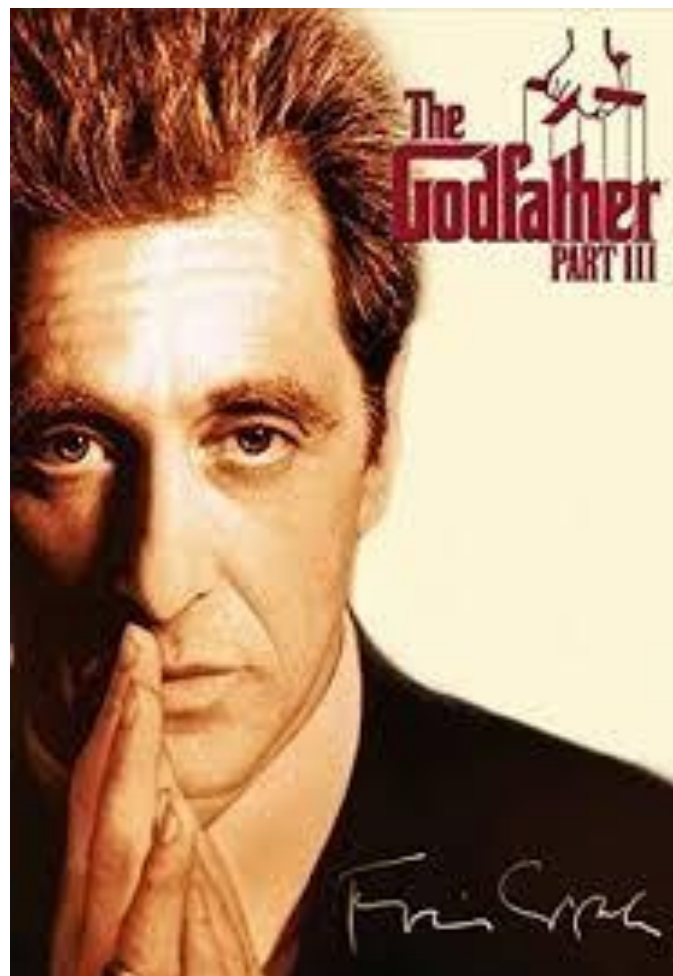


User	Horror	Sci-Fi	Humanity	Drama	...
Alice	0.3	2.1	6.1	4.7	...

Basic idea 1 (2/2)

Assumes that **latent factors** exist in users/items

User	God father	Terminator	Money game	Titanic	Back to the future	...	X-men
Alice	5	1	4	4	3	...	2



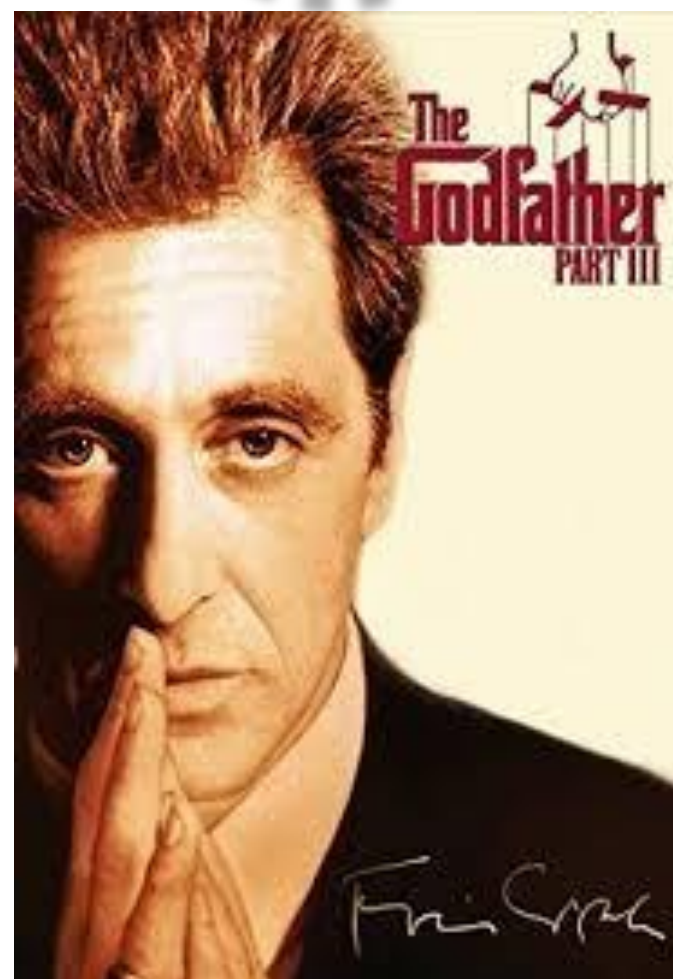
Latent factors (which cannot be observed)

User	Horror	Sci-Fi	Humanity	Drama	...
God father	2.3	0.4	4.9	5.7	...

Basic idea 2

Assumes that **rating scores** derive from **latent factors** of users and items

User	God father	Terminator	Money game	Titanic	Back to the future	...	X-men
Alice	5	1	4	4	3	...	2



User	Horror	Sci-Fi	Humanity	Drama	...
God father	2.3	0.4	4.9	5.7	...

×

User	Horror	Sci-Fi	Humanity	Drama	...
Alice	0.3	2.1	6.1	4.7	...

Summary of matrix factorization

$$\begin{bmatrix} 3 & \square & \dots & 3 \\ 5 & & & \\ & \square & & \\ 2 & & \dots & 1 \end{bmatrix} \approx \begin{bmatrix} 1.3 & \dots & 0.7 \\ & & \\ 0.2 & \dots & 4.9 \end{bmatrix} \times \begin{bmatrix} 0.2 & 1.1 & \dots & 0.9 \\ & & & \\ 3.1 & \dots & 0.7 & 1 \end{bmatrix}$$

Rating matrix = R
 (m users × n items)
 (□ = unknown scores)

User
 Latent user matrix
 (m users × k latent factors)

Item
 Latent item matrix
 (k latent factors × n items)

$$R \approx P \times Q^T$$

- Rating matrix can be decomposed to **latent factors** of users and items
- The dimension of latent factors (vectors) is **much less** than the number of users and items ($k \ll m, n$)

Prediction using matrix factorization

$$\begin{bmatrix} 3 & \boxed{2} & \dots & 3 \\ 5 & & \ddots & \boxed{5} \\ & \boxed{4} & & \\ 2 & \dots & & 1 \end{bmatrix} = \begin{bmatrix} 1.3 & \dots & 0.7 \\ & \ddots & \\ 0.2 & \dots & 4.9 \end{bmatrix} \times \begin{bmatrix} 0.2 & 1.1 & \dots & 0.9 \\ & \ddots & & \\ 3.1 & \dots & 0.7 & 1 \end{bmatrix}$$

Predicted rating matrix = R^*

Latent user matrix

Latent item matrix

$$R^* = P \times Q^T$$

If we obtain latent user/item matrix, we can **predict unknown scores** by multiplying the two latent matrix

How to obtain latent user/item matrix?

SVD: singular value decomposition

- A famous linear algebra technique for matrix decomposition
- It is often used for dimensionality reduction
- SVD delivers essentially the same result as PCA does

$$\begin{array}{c} X \\ m \times n \text{ matrix} \end{array} = \begin{array}{c} m \times m \\ \text{matrix} \\ \downarrow \\ U \end{array} \times \begin{array}{c} \Sigma \\ \uparrow \\ \text{Rectangular diagonal matrix} \\ \text{(diagonal values are called as singular values)} \end{array} \times \begin{array}{c} n \times n \\ \text{matrix} \\ \downarrow \\ V^T \end{array}$$

Rectangular diagonal matrix

(diagonal values are called as singular values)



Example of SVD

$$X = U \times \Sigma \times V^T$$

$$X = \begin{bmatrix} 1 & 3 & 3 & 3 & 0 \\ 2 & 4 & 2 & 2 & 4 \\ 1 & 3 & 3 & 5 & 1 \\ 4 & 5 & 2 & 3 & 3 \\ 1 & 1 & 5 & 2 & 1 \end{bmatrix}$$

$$U = \begin{bmatrix} -0.369 & -0.325 & 0.282 & 0.343 & 0.749 \\ -0.459 & 0.448 & -0.339 & -0.584 & 0.364 \\ -0.468 & -0.359 & 0.544 & -0.459 & -0.381 \\ -0.563 & 0.491 & 0.07 & 0.562 & -0.348 \\ -0.341 & -0.569 & -0.71 & 0.118 & -0.202 \end{bmatrix}$$

$$\Sigma = \begin{bmatrix} 13.368 & 0 & 0 & 0 & 0 \\ 0 & 4.708 & 0 & 0 & 0 \\ 0 & 0 & 2.792 & 0 & 0 \\ 0 & 0 & 0 & 1.586 & 0 \\ 0 & 0 & 0 & 0 & 0.904 \end{bmatrix}$$

$$V = \begin{bmatrix} -0.325 & 0.341 & -0.101 & 0.682 & -0.55 \\ -0.562 & 0.345 & 0.273 & 0.153 & 0.684 \\ -0.468 & -0.642 & -0.577 & 0.125 & 0.141 \\ -0.504 & -0.327 & 0.601 & -0.324 & -0.416 \\ -0.324 & 0.496 & -0.47 & -0.626 & -0.189 \end{bmatrix}$$



We can approximate a given matrix by focusing on the **largest singular values** and the vectors of U and V which correspond to the singular values

$$U = \begin{bmatrix} -0.369 & -0.325 & 0.282 & 0.343 & 0.749 \\ -0.459 & 0.448 & -0.339 & -0.584 & 0.364 \\ -0.468 & -0.359 & 0.544 & -0.459 & -0.381 \\ -0.563 & 0.491 & 0.07 & 0.562 & -0.348 \\ -0.341 & -0.569 & -0.71 & 0.118 & -0.202 \end{bmatrix}$$

$$\Sigma = \begin{bmatrix} 13.368 & 0 & 0 & 0 & 0 \\ 0 & 4.708 & 0 & 0 & 0 \\ 0 & 0 & 2.792 & 0 & 0 \\ \text{Largest singular values} & & & 1.586 & 0 \\ 0 & 0 & 0 & 0 & 0.904 \end{bmatrix}$$

$$V = \begin{bmatrix} -0.325 & 0.341 & -0.101 & 0.682 & -0.55 \\ -0.562 & 0.345 & 0.273 & 0.153 & 0.684 \\ -0.468 & -0.642 & -0.577 & 0.125 & 0.141 \\ -0.504 & -0.327 & 0.601 & -0.324 & -0.416 \\ -0.324 & 0.496 & -0.47 & -0.626 & -0.189 \end{bmatrix}$$



We can approximate a given matrix by focusing on the **largest singular values** and the vectors of U and V which correspond to the singular values

$$\begin{aligned}
 U &= \begin{bmatrix} -0.369 & -0.325 & 0.282 & 0.343 & 0.749 \\ -0.459 & 0.448 & -0.339 & -0.584 & 0.364 \\ -0.468 & -0.359 & 0.544 & -0.459 & -0.381 \\ -0.563 & 0.491 & 0.07 & 0.562 & -0.348 \\ -0.341 & -0.569 & -0.71 & 0.118 & -0.202 \end{bmatrix} \\
 \Sigma &= \begin{bmatrix} 13.368 & 0 & 0 & 0 & 0 \\ 0 & 4.708 & 0 & 0 & 0 \\ 0 & 0 & 2.792 & 0 & 0 \\ 0 & 0 & 0 & 1.555 & 0 \\ 0 & 0 & 0 & 0 & 0.904 \end{bmatrix} \\
 V &= \begin{bmatrix} -0.325 & 0.341 & -0.101 & 0.682 & -0.55 \\ -0.562 & 0.345 & 0.273 & 0.153 & 0.684 \\ -0.468 & -0.642 & -0.577 & 0.125 & 0.141 \\ -0.504 & -0.327 & 0.601 & -0.324 & -0.55 \\ -0.324 & 0.496 & -0.47 & -0.626 & -0.15 \end{bmatrix}
 \end{aligned}$$

Ignore unimportant values!

We can approximate a given matrix by focusing on the **largest singular values** and the vectors of U and V which correspond to the singular values

$$U_2 = \begin{bmatrix} -0.369 & -0.325 \\ -0.459 & 0.448 \\ -0.468 & -0.359 \\ -0.563 & 0.491 \\ -0.341 & -0.569 \end{bmatrix}$$

$$\Sigma_2 = \begin{bmatrix} 13.368 & 0 \\ 0 & 4.708 \end{bmatrix}$$

$$V_2 = \begin{bmatrix} -0.325 & 0.341 \\ -0.562 & 0.345 \\ -0.468 & -0.642 \\ -0.504 & -0.327 \\ -0.324 & 0.496 \end{bmatrix}$$

We can approximate a given matrix by focusing on the **largest singular values** and the vectors of U and V which correspond to the singular values

$$U_2 \times \Sigma_2 \times V_2^T = \begin{bmatrix} 1.08 & 2.24 & 3.29 & 2.98 & 0.84 \\ 2.72 & 4.17 & 1.52 & 2.41 & 3.04 \\ 1.46 & 2.93 & 4.02 & 3.71 & 1.19 \\ 3.24 & 5.03 & 2.04 & 3.04 & 3.59 \\ 0.57 & 1.64 & 3.86 & 3.18 & 0.15 \end{bmatrix} \approx \begin{bmatrix} 1 & 3 & 3 & 3 & 0 \\ & 4 & 2 & 2 & 4 \\ 1 & 3 & 3 & 5 & 1 \\ 4 & 5 & 2 & 3 & 3 \\ 1 & 1 & 5 & 2 & 1 \end{bmatrix} = X$$



Apply SVD for recommender systems (1/2)

Regarded as zero

	Item1	Item2	Item3	Item4	Item5
Alice	1	3	3	3	?
User1	2	4	2	2	4
User2	1	3	3	5	1
User3	4	5	2	3	3
User4	1	1	5	2	1

To matrix

$$\begin{bmatrix} 1 & 3 & 3 & 3 & 0 \\ 2 & 4 & 2 & 2 & 4 \\ 1 & 3 & 3 & 5 & 1 \\ 4 & 5 & 2 & 3 & 3 \\ 1 & 1 & 5 & 2 & 1 \end{bmatrix} = X$$



Apply SVD for recommender systems (2/2)

Step 1 Run SVD

$$X = U \times \Sigma \times V^T$$

Step 2

Focus on important features

$$\begin{array}{c} \text{---} \\ \downarrow \end{array} \begin{array}{c} \text{---} \\ \downarrow \end{array} \begin{array}{c} \text{---} \\ \downarrow \end{array}$$
$$U_2 \times \Sigma_2 \times V_2^T$$

||

Step 3 Multiply three matrix

$$\begin{bmatrix} 1.08 & 2.24 & 3.29 & 2.98 & \mathbf{0.84} \\ 2.72 & 4.17 & 1.52 & 2.41 & 3.04 \\ 1.46 & 2.93 & 4.02 & 3.71 & 1.19 \\ 3.24 & 5.03 & 2.04 & 3.04 & 3.59 \\ 0.57 & 1.64 & 3.86 & 3.18 & 0.15 \end{bmatrix}$$

Step 4

Check the values
which were zero
before running SVD



University of
East London

Singular Value Decomposition for Collaborative Filtering

Whiteboard

- Optimization problem
- Equivalence to Unconstrained Matrix Factorization (fully specified, no regularization)



MATRIX FACTORIZATION

Matrix Factorization

- Many different ways of factorizing a matrix
- We'll consider :
 1. Unconstrained Matrix Factorization
- MF is just another example of a **common recipe**:
 1. define a model
 2. define an objective function
 3. optimize with SGD



Example: MF for Netflix Problem

		NERO	JULIUS CAESAR	CLEOPATRA	SLEEPLESS IN SEATTLE	PRETTY WOMAN	CASABLANCA
HISTORY	1	1	1	1	0	0	0
	2	1	1	1	0	0	0
	3	1	1	1	0	0	0
BOTH	4	1	1	1	1	1	1
	5	-1	-1	-1	1	1	1
ROMANCE	6	-1	-1	1	1	1	1
	7	-1	-1	-1	1	1	1

 $\approx$

		HISTORY	ROMANCE
1	1	1	0
2	1	1	0
3	1	1	0
4	1	1	1
5	-1	1	1
6	-1	1	1
7	-1	1	1

 $\times$

		NERO	JULIUS CAESAR	CLEOPATRA	SLEEPLESS IN SEATTLE	PRETTY WOMAN	CASABLANCA
HISTORY	1	1	1	1	0	0	0
	2	1	1	1	0	0	0
	3	1	1	1	0	0	0
BOTH	4	1	1	1	1	1	1
	5	-1	-1	-1	1	1	1
ROMANCE	6	-1	-1	1	1	1	1
	7	-1	-1	-1	1	1	1

 V^T

(a) Example of rank-2 matrix factorization

		NERO	JULIUS CAESAR	CLEOPATRA	SLEEPLESS IN SEATTLE	PRETTY WOMAN	CASA LANCA
HISTORY	1	0	0	0	0	0	0
	2	0	0	0	0	0	0
	3	0	0	0	0	0	0
BOTH	4	0	0	1	0	0	0
	5	0	0	1	0	0	0
ROMANCE	6	0	0	1	0	0	0
	7	0	0	1	0	0	0

(b) Residual matrix

Unconstrained Matrix Factorization

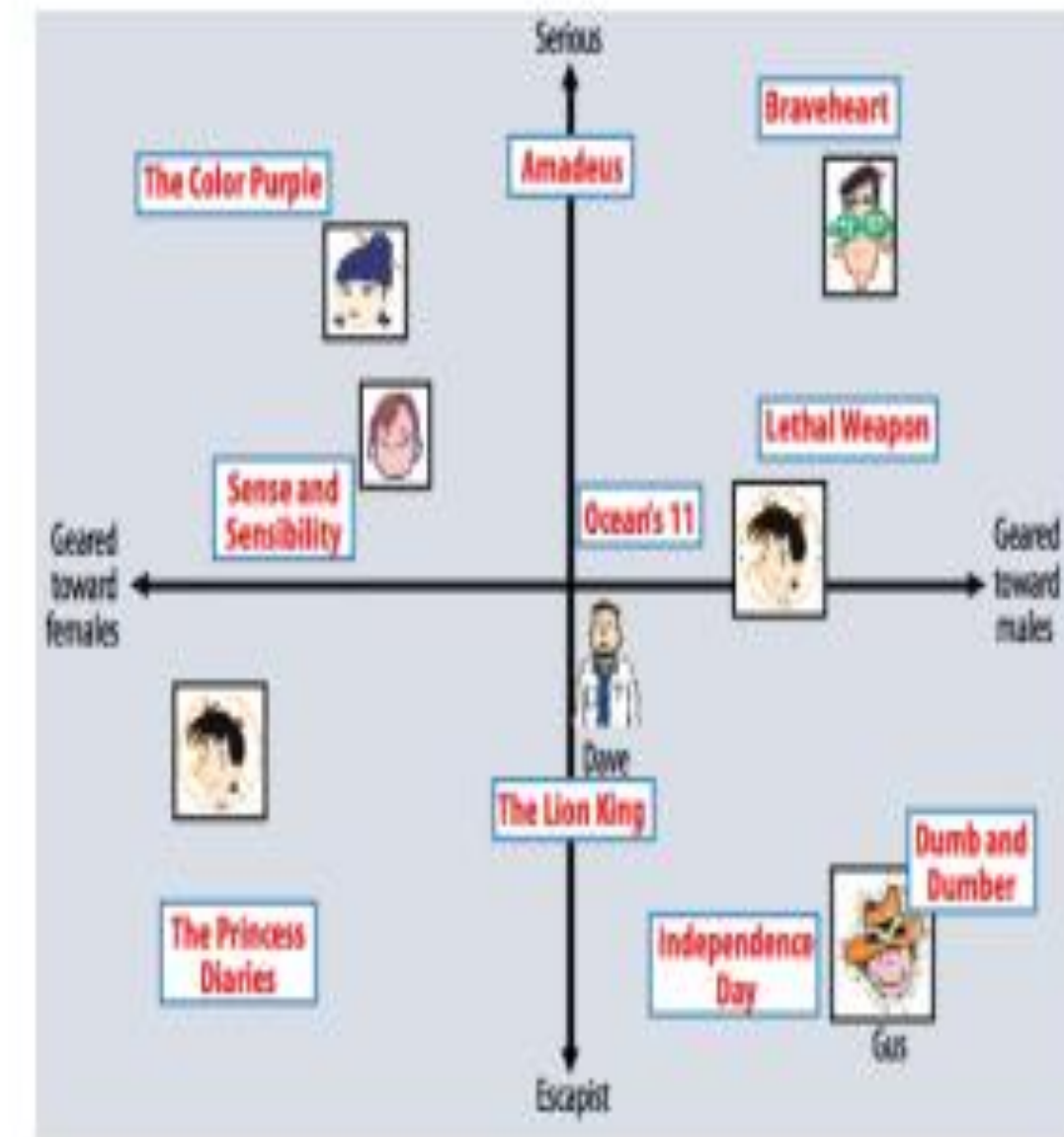
Derive a block coordinate descent algorithm for the Unconstrained Matrix Factorization problem.

- User vectors:
 $\mathbf{w}_u \in \mathbb{R}^r$
- Item vectors:
 $\mathbf{h}_i \in \mathbb{R}^r$
- Rating prediction:
 $v_{ui} = \mathbf{w}_u^T \mathbf{h}_i$
- Set of non-missing entries
 $\mathcal{Z} = \{(u, i) : v_{ui} \text{ is observed}\}$
- Objective:
$$\operatorname{argmin}_{\mathbf{w}, \mathbf{h}} \sum_{(u, i) \in \mathcal{Z}} (v_{ui} - \mathbf{w}_u^T \mathbf{h}_i)^2$$

Matrix Factorization (with vectors)

- Regularized Objective:

$$\begin{aligned} \operatorname{argmin}_{\mathbf{w}, \mathbf{h}} \sum_{(u, i) \in \mathcal{Z}} (v_{ui} - \mathbf{w}_u^T \mathbf{h}_i)^2 \\ + \lambda \left(\sum_i \|\mathbf{w}_i\|^2 + \sum_u \|\mathbf{h}_u\|^2 \right) \end{aligned}$$



Figures from Koren et al. (2009)



Matrix Factorization (with vectors)

- Regularized Objective:

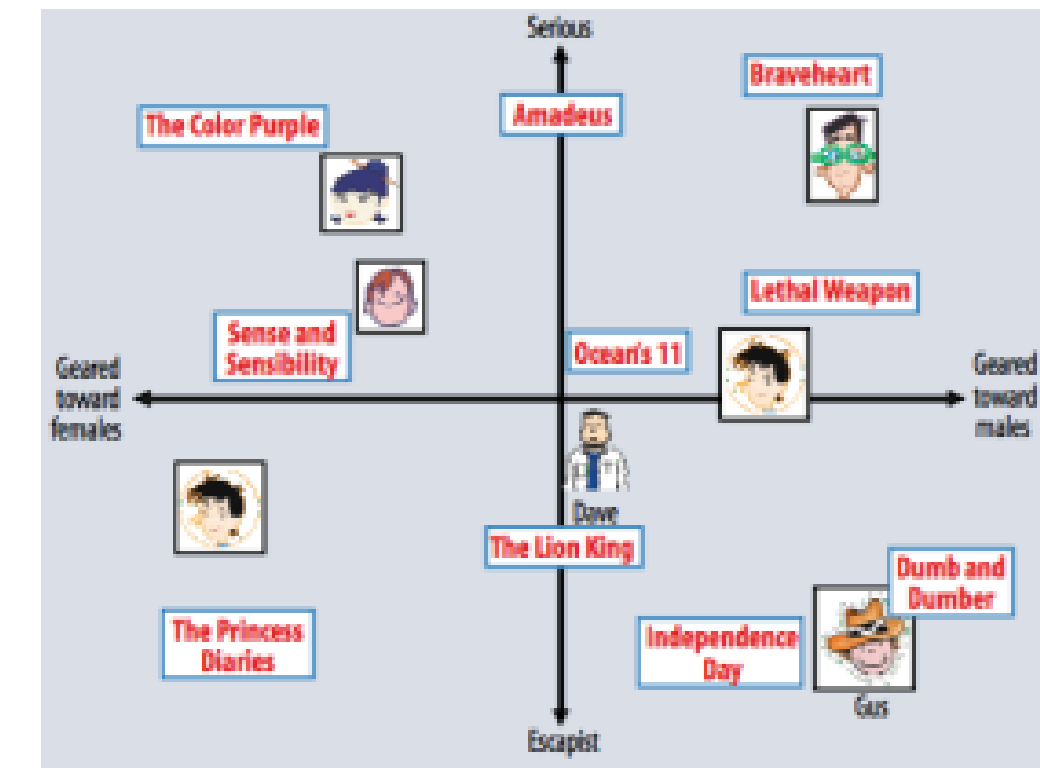
$$\begin{aligned} \operatorname{argmin}_{\mathbf{w}, \mathbf{h}} \sum_{(u,i) \in \mathcal{Z}} (v_{ui} - \mathbf{w}_u^T \mathbf{h}_i)^2 \\ + \lambda \left(\sum_i \|\mathbf{w}_i\|^2 + \sum_u \|\mathbf{h}_u\|^2 \right) \end{aligned}$$

- SGD update for random (u,i):

$$e_{ui} \leftarrow v_{ui} - \mathbf{w}_u^T \mathbf{h}_i$$

$$\mathbf{w}_u \leftarrow \mathbf{w}_u + \gamma(e_{ui} \mathbf{h}_i - \lambda \mathbf{w}_u)$$

$$\mathbf{h}_i \leftarrow \mathbf{h}_i + \gamma(e_{ui} \mathbf{w}_u - \lambda \mathbf{h}_i)$$



Figures from Koren et al. (2009)



Matrix Factorization (with matrices)

- SGD

require that the loss can be written as

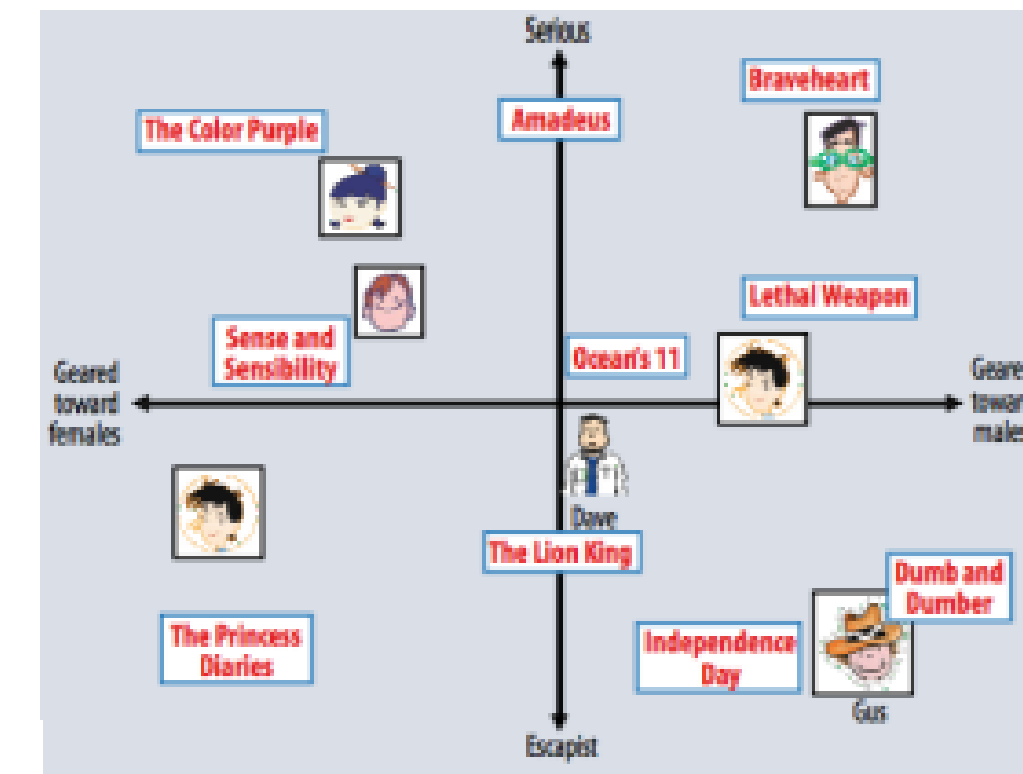
$$L = \sum_{(i,j) \in Z} l(\mathbf{V}_{ij}, \mathbf{W}_{i*}, \mathbf{H}_{*j})$$

Algorithm 1 SGD for Matrix Factorization

Require: A training set Z , initial values $\mathbf{W}_0$ and $\mathbf{H}_0$
while not converged **do** {step}
 Select a training point $(i, j) \in Z$ uniformly at random.
 $\mathbf{W}'_{i*} \leftarrow \mathbf{W}_{i*} - \epsilon_n N \frac{\partial}{\partial \mathbf{W}_{i*}} l(\mathbf{V}_{ij}, \mathbf{W}_{i*}, \mathbf{H}_{*j})$
 $\mathbf{H}_{*j} \leftarrow \mathbf{H}_{*j} - \epsilon_n N \frac{\partial}{\partial \mathbf{H}_{*j}} l(\mathbf{V}_{ij}, \mathbf{W}_{i*}, \mathbf{H}_{*j})$
 $\mathbf{W}_{i*} \leftarrow \mathbf{W}'_{i*}$
end while

step size

Figure from Gemulla et al. (2011)



Figures from Koren et al. (2009)

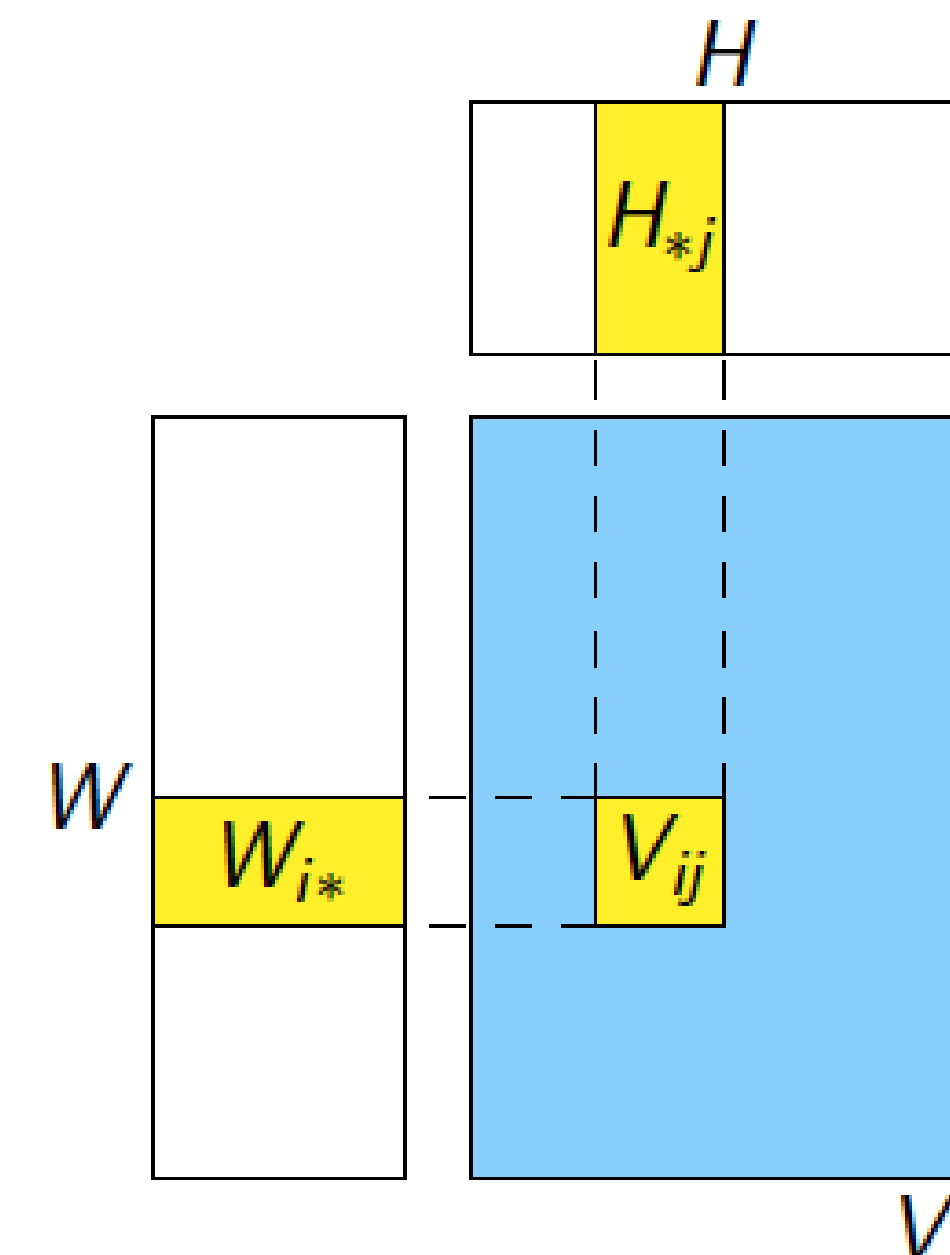


Figure from Gemulla et al. (2011)



University of
East London

Matrix Factorization (with matrices)

- User vectors:

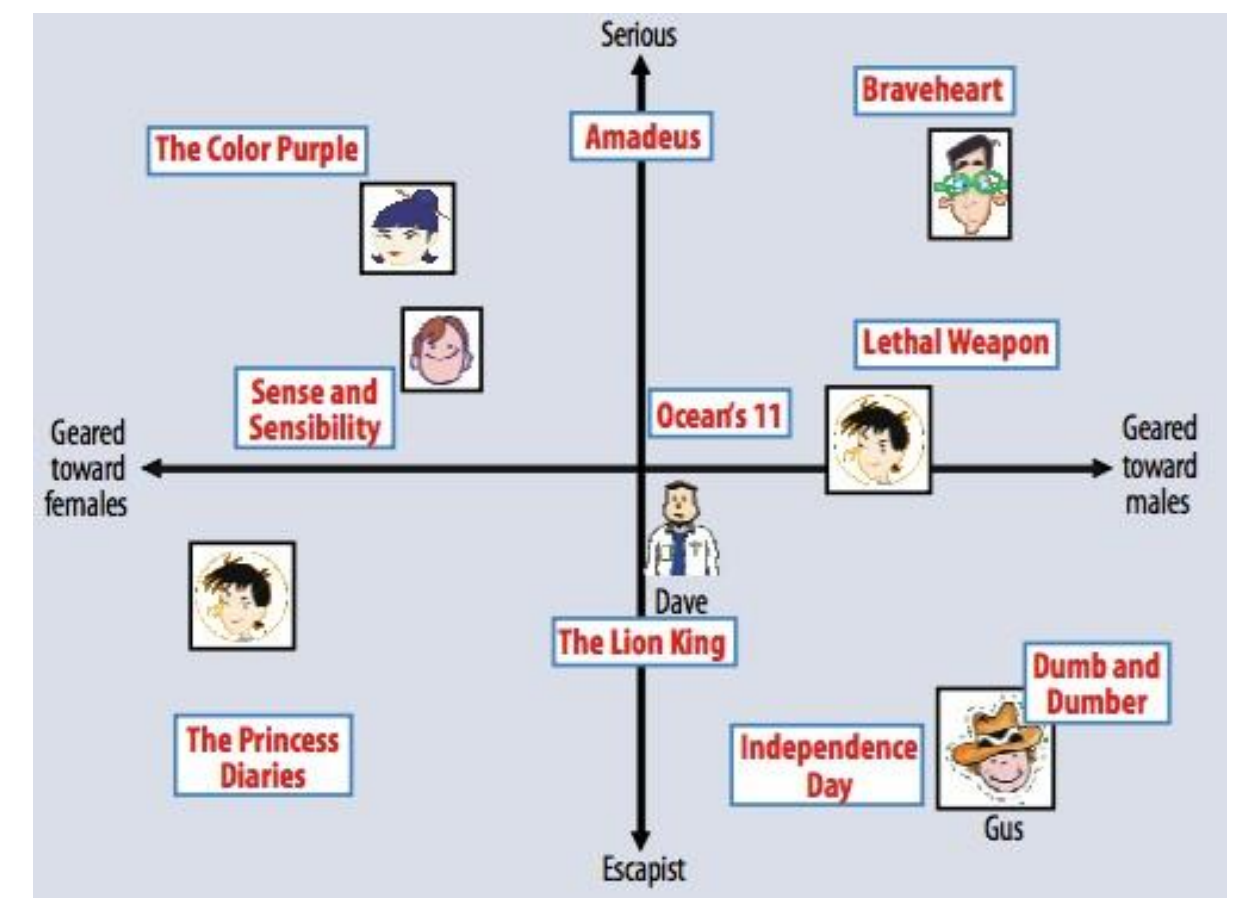
$$(W_u^*)^T \in \mathbb{R}^r$$

- Item vectors:

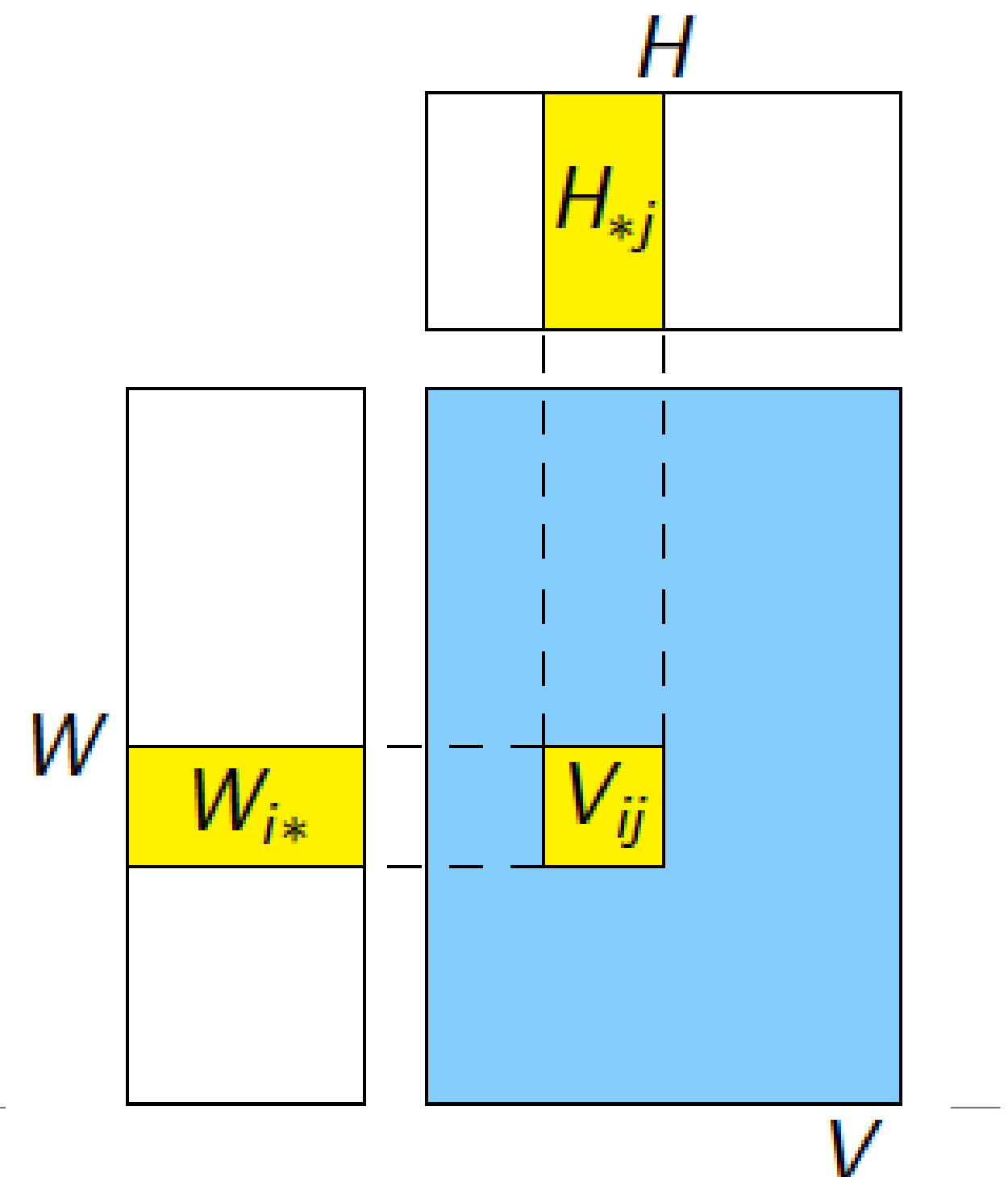
$$H_i \in \mathbb{R}^r$$

- Rating prediction:

$$\begin{aligned} V_{ui} &= W_u^* H_{*i} \\ &= [WH]_{ui} \end{aligned}$$



Figures from Koren et al. (2009)



Alternating Least Squares (ALS)

A popular approach for this is **matrix factorization**, where we fix a relatively small number k (e.g. $k \approx 10$), and summarize each user u with a k dimensional vector x_u , and each item i with a k dimensional vector y_i . These vectors are referred to as *factors*. Then, to predict user u 's rating for item i , we simply predict $r_{ui} \approx x_u^T y_i$. This can be put in matrix form: Let $x_1, \dots, x_n \in \mathbb{R}^k$ be the factors for the users, and $y_1, \dots, y_m \in \mathbb{R}^k$ the factors for the items. The $k \times n$ user matrix X , and the $k \times m$ item matrix Y are then defined by:

$$X = \begin{bmatrix} | & & | \\ x_1 & \cdots & x_n \\ | & & | \end{bmatrix}, Y = \begin{bmatrix} | & & | \\ y_1 & \cdots & y_m \\ | & & | \end{bmatrix}$$

Our goal is then to estimate the complete ratings matrix $R \approx X^T Y$. We can formulate this problem as an optimization problem in which we aim to minimize an objective function and find optimal X and Y . In particular, we aim to minimize the least squares error of the observed ratings (and regularize):

$$\min_{X,Y} \sum_{r_{ui} \text{ observed}} (r_{ui} - x_u^T y_i)^2 + \lambda \left(\sum_u \|x_u\|^2 + \sum_i \|y_i\|^2 \right) \quad (1)$$

Notice that this objective is non-convex (because of the $x_u^T y_i$ term); in fact it's NP-hard to optimize. Gradient descent can be used as an approximate approach here, however it turns out to be slow and costs lots of iterations. Note however, that if we fix the set of variables X and treat them as constants, then the objective is a convex function of Y and vice versa. Our approach will

Alternating Least Squares (ALS)

- ❑ Our approach will 1 therefore be to fix Y and optimize X , then fix X and optimize Y , and repeat until convergence.
- ❑ This approach is known as ALS(Alternating Least Squares).
- ❑ For our objective function, the alternating least squares algorithm is as follows:

Prediction: $\hat{r}_{ui} \approx x_u^T y_i$

Alternating Least Squares (ALS)

Algorithm 1 ALS for Matrix Completion

Initialize X, Y

repeat

 for $u = 1 \dots n$ do

$$x_u = \left(\sum_{r_{ui} \in r_{u*}} y_i y_i^T + \lambda I_k \right)^{-1} \sum_{r_{ui} \in r_{u*}} r_{ui} y_i$$

 end for

 for $i = 1 \dots m$ do

$$y_i = \left(\sum_{r_{ui} \in r_{*i}} x_u x_u^T + \lambda I_k \right)^{-1} \sum_{r_{ui} \in r_{*i}} r_{ui} x_u$$

 end for

until convergence

Matrix Factorization

Comparison of Optimization Algorithms

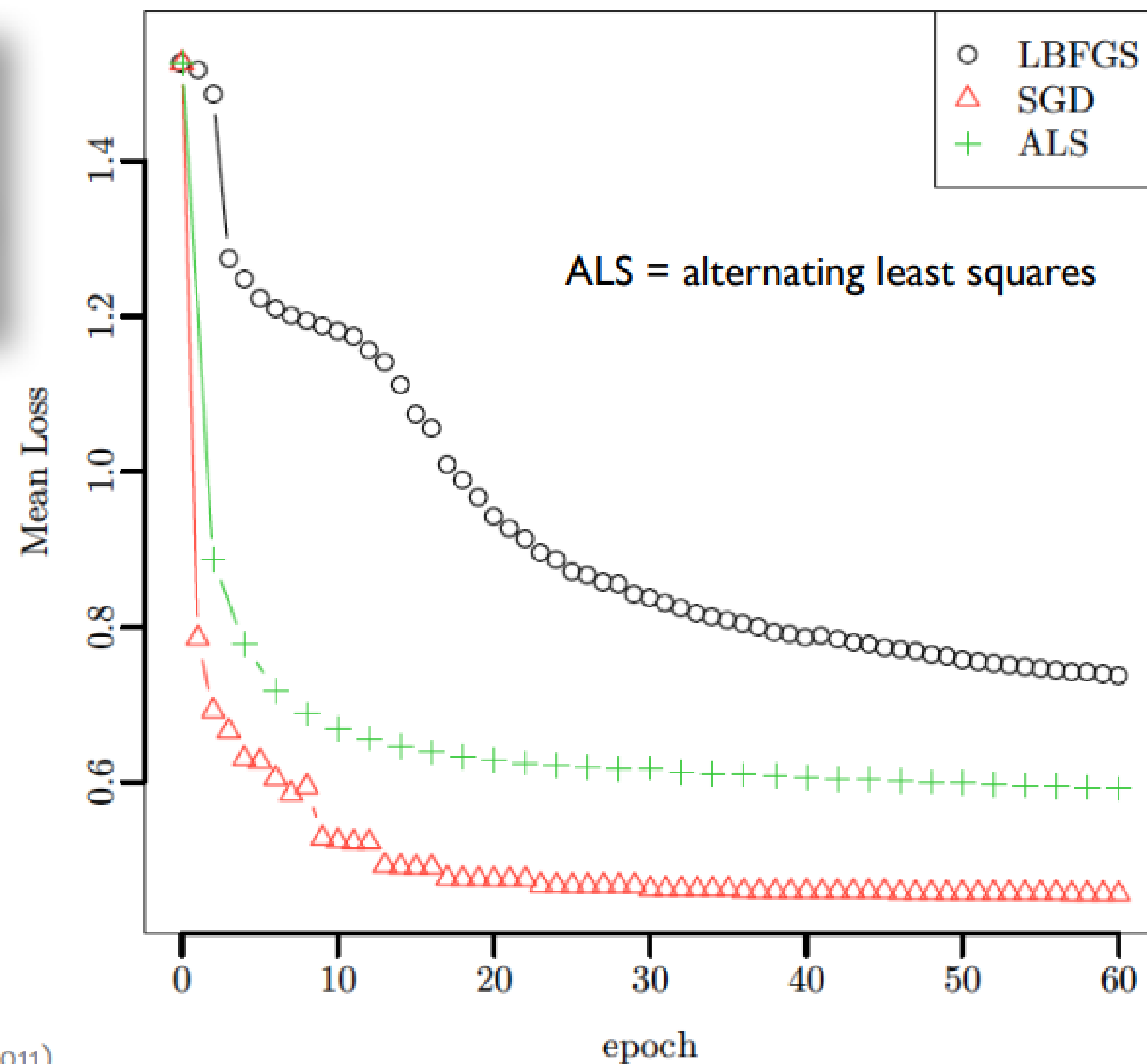
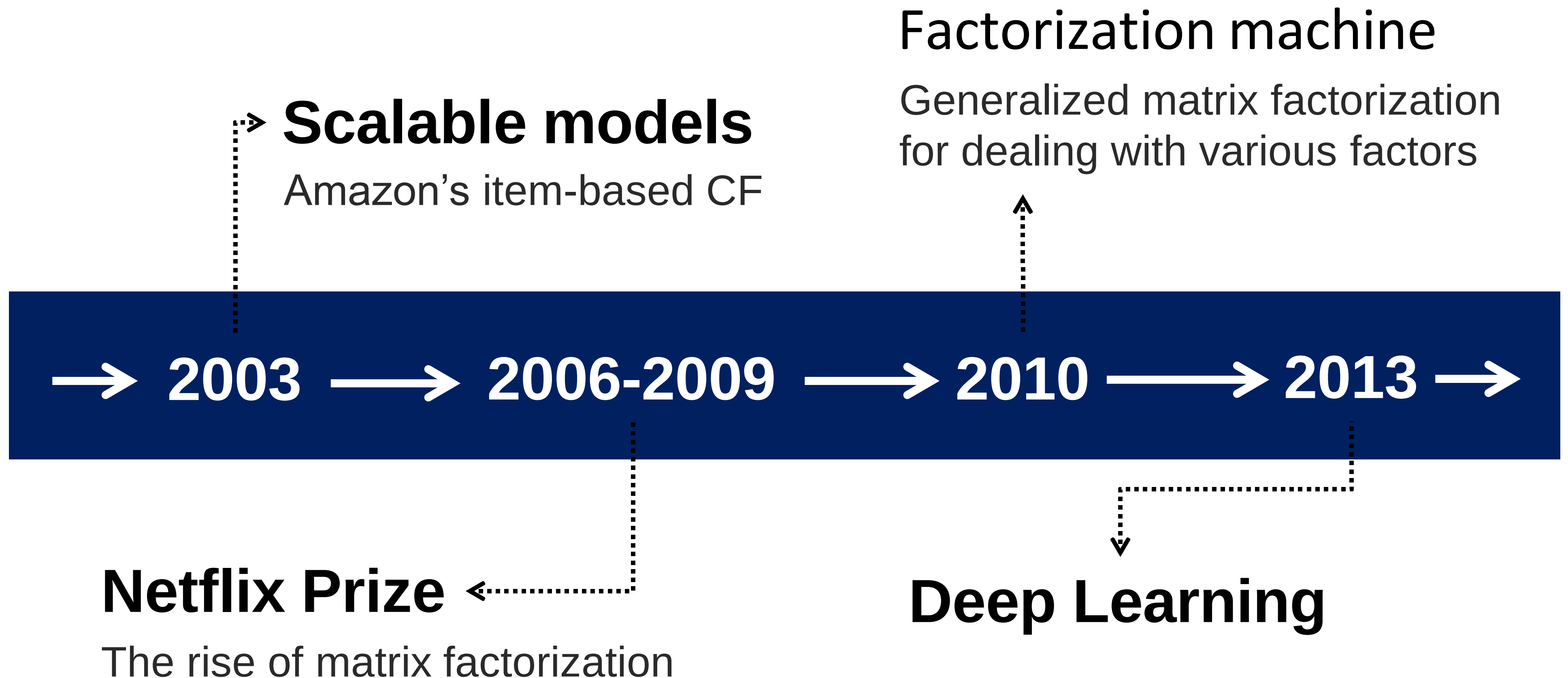


Figure from Gemulla et al. (2011)



Various approaches have been developed ...



Summary

- Recommender systems solve many **real-world** (*large-scale) **problems**
- Collaborative filtering by Matrix Factorization (MF) is an **efficient** and **effective** approach



References and Online Resources

- LinkedIn: <https://www.linkedin.com/learning/comptia-data-plus-da0-001-cert-prep-domain-3-0-data-analysis/data-analysis?u=56744289>
- <http://stanford.edu/~rezab/classes/cme323/S15/notes/lec14.pdf>
- Interview Tips: <https://www.linkedin.com/advice/1/what-best-ways-prepare-data-analysis-interviews>

